DIVIDE TWO INTEGERS

```
class Solution(object):
    def divide(self, dividend, divisor):
        """
        :type dividend: int
        :type divisor: int
        :rtype: int
        """
        INT_MAX = 2**31 - 1
        INT_MIN = -2**31

        if dividend == INT_MIN and divisor == -1:
            return INT_MAX

        a = abs(dividend)
        b = abs(divisor)
        result = 0

        while a >= b:
            temp = b
            multiple = 1
            while a >= (temp << 1):
                temp <<= 1
                multiple <<= 1
            a -= temp
            result += multiple
        return result if (dividend > 0) == (divisor > 0) else -result
```

SEARCH INSERT POSITION

```
class Solution(object):
    def searchInsert(self, nums, target):
        """
        :type nums: List[int]
        :type target: int
        :rtype: int
        """
        nums.sort()
        s=0
        e=len(nums)-1
        while s<=e:
            mid=(s+e)//2
            if nums[mid]==target:
                return mid
            elif nums[mid]<target:
                s=mid+1
            else:
                e=mid-1
        return s
```

Length Of Last Word

```
class Solution(object):
    def lengthOfLastWord(self, s):
        """ :type s: str
        :rtype: int
        """
 words = s.strip().split()
        return len(words[-1]) if words else 0
```

Plus One

```
class Solution(object):
    def plusOne(self, digits):
        """
        :type digits: List[int]
        :rtype: List[int]
        """
        # Traverse the list from the last element to the first
        for i in range(len(digits)-1, -1, -1):
            # If the current digit is 9, set it to 0
            if digits[i] == 9:
                digits[i] = 0
            else:
                # If the current digit is not 9, increment it by 1 and return the list
                digits[i] = digits[i] + 1
                return digits
        # If all digits are 9, prepend 1 to the list
        return [1] + digits
```

AddBinary

```
class Solution(object):
    def addBinary(self, a, b):


        return str(bin(int(a,2)+int(b,2)))[2:]
```

Sqrt(X)

```
class Solution(object):
    def mySqrt(self, x):


        low = 1
        high = x
        p = 1


        if x < 2:
            return x
        while low <= high:
            mid = (low + high) // 2
            if mid * mid == x:
                return mid
            elif mid * mid < x:
                p = mid
                low = mid + 1
            else:
                high = mid - 1
        return p
```

ClimbStairs

```
class Solution(object):
    def climbStairs(self, n):
        """
        :type n: int
        :rtype: int
        """
        if n==1:
            return 1


        dp=[0]*(n+1)
        dp[0]=1
        dp[1]=1
        for i in range(2,n+1):
            dp[i]=dp[i-1]+dp[i-2]
        return dp[n]
```

SameTree

```
class Solution(object):
    def isSameTree(self, p, q):
        if not p and not q:
            return True
        if not p or not q:
            return False
        return p.val == q.val and self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)
```

Symmetric Tree

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution(object):
    def isSymmetric(self, root):
        """
        :type root: Optional[TreeNode]
        :rtype: bool
        """
        if not root:
            return True
        return self.ismirror(root.left,root.right)
    def ismirror(self,left,right):
        if not left and not right:
            return True
        if not left or not right:
            return False
        return(
            left.val==right.val and
            self.ismirror(left.left,right.right) and
            self.ismirror(left.right,right.left)
        )

        return False
```

Palindrome

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        new_str = "".join([char for char in s if char.isalnum()]).lower()
        return new_str == new_str[::-1]
```

singleNumber

```
class Solution(object):
    def singleNumber(self, nums):
        # Initialize the unique number...
        uniqNum = 0;
        # TRaverse all elements through the loop...
        for idx in nums:
            # Concept of XOR...
            uniqNum ^= idx;
        return uniqNum;      # Return the unique number...
```

countPrimes

```
class Solution(object):
    def countPrimes(self, n):
        if n <= 2:
            return 0

        is_prime = [True] * n
        is_prime[0] = is_prime[1] = False

        i = 2
        while i * i < n:
            if is_prime[i]:
                for j in range(i * i, n, i):
                    is_prime[j] = False
            i += 1

        return sum(is_prime)
```

PowerOfTwo

```
class Solution(object):
    def isPowerOfTwo(self, n):
        """
:type n: int
        :rtype: bool
        """
     return n > 0 and (n & (n - 1)) == 0
```

hammingWeight

```
class Solution {
  public int hammingWeight(int n) {
    int count = 0;
    while (n != 0) {
      count += n & 1;
      n >>= 1;
    }
    return count;
  }
}
```

MissingPositive

```
class Solution(object):
  def firstMissingPositive(self, nums):
    n = len(nums)
    for i, num in enumerate(nums):
      if num <= 0:
        nums[i] = n + 1
    for num in nums:
      num = abs(num)
      if num <= n:
        nums[num - 1] = -abs(nums[num - 1])
    for i, num in enumerate(nums):
      if num > 0:
        return i + 1

    return n + 1
```

missingInteger

```
class Solution:
    def missingInteger(self, nums):
        # Step 1: find longest sequential prefix sum
        prefix_sum = nums[0]
        i = 1
        while i < len(nums) and nums[i] == nums[i-1] + 1:
            prefix_sum += nums[i]
            i += 1

        # Step 2: find smallest missing >= prefix_sum
        num_set = set(nums)
        x = prefix_sum
        while x in num_set:
            x += 1

        return x
```

partition list

```
class Solution:
    def partition(self, head, x):
        # Two dummy heads for two lists
        smaller = ListNode(0)
        greater = ListNode(0)

        # Pointers to build the lists
        s = smaller
        g = greater

        # Go through all nodes
        while head:
            if head.val < x:
                s.next = head
                s = s.next
            else:
                g.next = head
                g = g.next
            head = head.next

        # Connect smaller list with greater list
        s.next = greater.next
        g.next = None

        # Return the start of the smaller list
        return smaller.nexteturn x
```